(Hussain Durrani - 0937)

Artificial Intelligence - Assignment # 01

Question No. 1 :-

-: PEAS Description :-

Ans:- PEAS stands for Performance Measure, Environment, Actuators, and Sensors. It describes the task environment of an intelligent agent.

PEAS Components:

- **P-Performace Measure :-**

1) Deliver packages on time, and also safely.
2) Avoid collisions and accidents.
3) Minimize battery consumption.
4) Deliver packages to the correct destination.

- **E-Environment :-**

1) University buidings and corridors.
2) Campus roads and pathways.
3) Students, teachers, and other pedestrians.
4) Obstacles and changing weather conditions.

- **A-Actuators :-**

1) Wheels and motors for movement.

CamScanner

2) Steering and breaking system.
3) Robotic arm or compartment lock of delivery package.
4) Speaker or display for communication.
~~5) Camera or sensors for detecting obstacles.~~
~~6) GPS or navigation system for finding delivery location within the building.~~

- **S- Sensors :-**

1) Cameras for detecting obstacles.
2) GPS for location tracking.
3) Ultrasonic sensors or LiDAR for measuring distances.
4) Battery sensors for monitoring power.

Explanation:-

The robot uses sensors to observe its surroundings, processes the information using AI, and performs actions through actuators. Its performance is measured by delivery accuracy, safety, speed, and energy efficiency.

CamScanner

Question No 2:-

:- Environment Classification :-

	Environment	Classification	Justification
i)	Chess	Fully Observable	Both players can observe the complete board and the positions of all pieces
ii)	Taxi Driving	Stochastic	Traffic, pedestrians, and other vehicles introduce uncertainty into the outcome of actions.
iii)	Crossword Puzzle	Static	The puzzle remains unchanged while the agent is solving it.
iv)	Chess	Discrete	Chess has a finite set of board positions, & clearly defined legal moves.
v)	Multiplayer Video Game	Multi-Agent	Multiplayer interacts and compete, and their action affect one another.
vi)	Medical Diagnosis System	Partially Observable	The system can't directly observe every internal condition of the patient

Question No. 3:-

:- BFS vs DFS :-

Breadth-first Search (BFS) and Depth-first Search (DFS) are uninformed search algorithms used to explore a search tree and find a goal state.

CamScanner

BFS explores nodes level by level, whereas DFS explores one branch deeply before backtracking.

Property	BFS	DFS
• Completeness	Yes, if branching factor (b) is finite	No, for infinite-depth search spaces.
• Optimality	Yes, if step costs are identical.	No.
• Time Complexity	$O(b^{d+1})$	$O(b^m)$
• Space Complexity	$O(b^{d+1})$	$O(bm)$
• Data Structure	Queue (FIFO)	Stack (LIFO)

Where:

- b = Branching factor (max. No. of children of a node)
- d = Depth of the shallowest goal.
- m = Maximum depth of the search tree.

BFS finds the shallowest goal and is optimal when step costs are equal, but it requires more memory.
DFS uses less memory but may get stuck exploring an infinite branch & does not guarantee an optimal solution.

CamScanner

Question No. 4 :-

-: DLS and IDS :-

Depth-Limited Search (DLS):-

DLS is a modified version of DFS in which a fixed depth limit L is applied. The algorithm explores nodes using DFS but does not expand beyond the specified depth limit.

Example:

If L=2, the algorithm searches only levels 0, 1, and 2.

Limitations:

1) DLS is not complete if the goal exists beyond the specified depth limit.
2) Choosing the correct depth limit is difficult when goal is unknown.
3) A small limit may prevent the algorithm from finding an existing solution.
4) DLS is not optimal because it may not find the least-cost path.

Iterative Deepening Search (IDS):-

IDS solves the depth-limit problem by repeatedly performing DLS with increasing depth limits.

CamScanner

Working:

Iteration	Depth Limit	Search
0	0	Search root node only
1	1	Search up to level 1
2	2	Search up to level 2
3	3	Search up to level 3
...	...	Continue until goal is found.

- **How IDS overcome DLS limitations:**

IDS increases the depth limit automatically if the goal is not found. Therefore, it does not require knowing the solution depth in advance. It inherits the memory advantage of DFS and the completeness property of BFS.

Time Complexity: $O(b^d)$
Space Complexity: $O(bd)$

IDS is useful when the goal depth is unknown because it searches progressively deeper while maintaining low memory requirements.

CamScanner

Question No. 5:-

-: Uninformed Searching – BFS and UCS :-

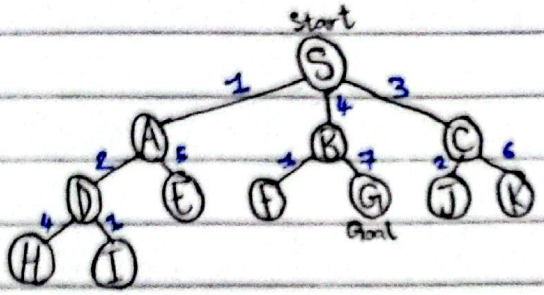
Given:

Start Node = S

Goal Node = G

Edge weights represent path costs.

- The graph is a tree, so each node has a unique path from S.
- We will expand children from 'left to right'.



(Part A)

-: Breadth-First Search (BFS) :-

BFS explores the shallowest nodes first. It uses a Queue (FIFO) and does not consider path costs when selecting the next node.

BFS Table:

Frontier (Queue)	Expand	Explored
S	ϕ	ϕ
A, B, C	S	S
B, C, D, E	A	S, A
C, D, E, F, G	B	S, A, B
D, E, F, G, J, K	C	S, A, B, C
E, F, G, J, K, H, I	D	S, A, B, C, D
F, G, J, K, H, I	E	S, A, B, C, D, E
G, J, K, H, I	F	S, A, B, C, D, E, F
J, K, H, I	G (Goal)	S, A, B, C, D, E, F, G

a) Traversal Order : S → A → B → C → D → E → F → G

b) Final Solution Path: S → B → G

c) Cost = 4 + 7 = 11

BFS visits nodes level by level and finds G at depth 2 through B. It finds the shallowest goal but does not guarantee the minimum path cost when step costs differ.

(Part B)

:- Uniform Cost Search (UCS) :-

UCS expands the node having the lowest cumulative path cost $g(n)$.

CamScanner

It uses a Priority Queue, where nodes are arranged in increasing order of their cumulative costs.

$g(n)$ = Total path cost from S to node n

UCS Table:

Frontier (Priority Queue)	Expanded	Explored
S(0)	ϕ	ϕ
A(1), C(3), B(4)	S	S
D(3), C(3), B(4), E(6)	A	S, A
C(3), B(4), I(4), E(6), H(7)	D	S, A, D
B(4), I(4), J(5), E(6), H(7), K(9)	C	S, A, D, C
I(4), F(5), J(5), E(6), H(7), K(9), G(11)	B	S, A, D, C, B
F(5), J(5), E(6), H(7), K(9), G(11)	I	S, A, D, C, B, I
J(5), E(6), H(7), K(9), G(11)	F	S, A, D, C, B, I, F
E(6), H(7), K(9), G(11)	J	S, A, D, C, B, I, F, J
H(7), K(9), G(11)	E	S, A, D, C, B, I, F, J, E
K(9), G(11)	H	S, A, D, C, B, I, F, J, E, H
G(11)	K	S, A, D, C, B, I, F, J, E, H, K
ϕ	G(Goal)	S, A, D, C, B, I, F, J, E, H, K, G

For equal costs, nodes are expanded in insertion order. The goal ~~Lets~~ test is applied when the node is removed from the priority queue.

a) Traversal Order: S → A → D → C → B → I → F → J → E → H → K → G

b) Final Optimal Path: $S \rightarrow B \rightarrow G_1$

c) Total Path Cost :

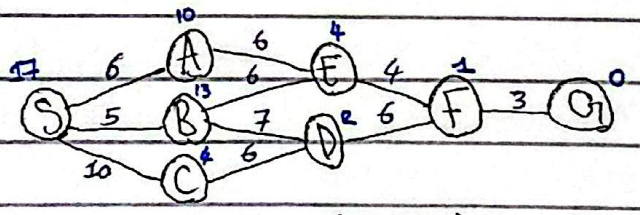
$$g(G_1) = g(S, B) + g(B, G_1)$$
$$= 4 + 7$$

Total Cost = 11

UCS finds the optimal path 'S → B → G_1' with a total cost of 11. It expands nodes according to cumulative path cost rather than depth.

Question No. 6 :-

:- Breadth-First Search (BFS) :-



BFS uses a Queue (FIFO) and explores nodes level by level. The algorithm does not consider edge costs or heuristic values.

Given:

Start Node = S

Goal Node = G_1

CamScanner

BFS Table:-

Frontier (Queue)	Expand	Explored
S	ϕ	ϕ
A, B, C	S	S
B, C, E	A	S, A
C, E, D	B	S, A, B
E, D	C	S, A, B, C
D, F	E	S, A, B, C, E
F	D	S, A, B, C, E, D
G	F	S, A, B, C, E, D, F
ϕ	G (Goal)	S, A, B, C, E, D, F, G

Traveral Order : S → A → B → C → E → D → F → G

Final Solution Path: S → A → E → F → G

Depth of Goal : 4

BFS starts from S and first explores its children A, B, and C. It then explores E and D, followed by F. Finally, it reaches G. Since BFS explores level by level, it finds a path with the minimum number of edges.

CamScanner

Question No. 7 :-

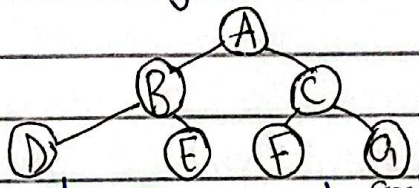
-: Iterative Deepening Search (IDS) :-

Iterative Deepening Search (IDS), also known as IDDFS, is an uninformed search algorithm that repeatedly performs Depth-Limited Search with increasing depth limits until the goal is found.

IDS starts at depth 0, the increases the depth limit to 1, 2, 3, and so on.

Working of IDS :-

Consider the following search tree:



#	Depth Limit	Traversal Order
0	0	A
1	1	A, B, C
2	2	A, B, D, E, C, F, G

Goal found at depth 2.

⇒ How IDS combines BFS and DFS

1) Advantage of DFS:

IDS uses depth-first exploration in every iteration, so it requires less memory than BFS.

CamScanner

2) Advantage of BFS:
IDS explores increasing depth limits, so it is complete for finite branching factors and finds the shallowest goal first.

3) Avoids the Depth-Limit Problem:
Unlike DLS, IDS does not depend on a single fixed depth limit. If the goal is not found, the depth limit is increased.

Is IDS Optimal?

Yes, IDS is optimal when all step costs are identical, because the shallowest goal will also have the mimimum path cost.
If step costs are different, IDS may find a shallow but expensive solution instead of a cheaper one.

Time Complexity: $O(b^d)$
Space Complexity: $O(bd)$

IDS Combines the completeness and shallowest-solution advantage of BFS with the memory efficiency of DFS.

CamScanner

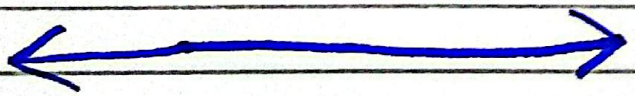
Question No. 8 :-

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity
DFS	No (in general)	No	$O(b^m)$	$O(bm)$
BFS	Yes	Yes, if step costs are identical	$O(b^{d+1})$	$O(b^{d+1})$
UCS	Yes, if step costs $\geq \epsilon > 0$	Yes	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$
IDDFS	Yes	Yes, if step costs are identical	$O(b^d)$	$O(bd)$

Where:

b = Branching Factor
d = Depth of the shallowest solution.
m = Maximum depth of the search tree.
C^* = Cost of the optimal solution.
ϵ = Minimum positive step cost.

DFS uses less memory but does not guarantee completeness or optimality in general. BFS finds the shallowest goal and is optimal for identical step costs. UCS expands nodes with the lowest cumulative cost and guarantees an optimal solution under positive step-cost assumptions. IDDFS combines the memory efficiency of DFS with the completeness and shallowest-solution advantage of BFS.



CamScanner